



Student Name : Pranay Mestry

Batch : CS2

Roll Number : CS31

Aim

To implement graph-based clustering using Minimum Spanning Tree (MST) on the Breast Cancer Wisconsin (Diagnostic) dataset in Python and visualize the resulting clusters.

Objectives

1. To understand the concept of graph-based clustering and Minimum Spanning Trees (MST).
2. To load and preprocess the Breast Cancer Wisconsin dataset (scaling features and handling metadata).
3. To construct a distance graph from tumor feature representations using Euclidean distance.
4. To compute the Minimum Spanning Tree using Prim's or Kruskal's algorithm (or via scipy/networkx).
5. To form $K=2$ clusters (Malignant vs. Benign) by removing the single $(K - 1)$ largest edge weight from the MST.
6. To evaluate the performance of the MST-based clustering model using evaluation metrics.

Relevant Theory

Graph-Based Clustering & Minimum Spanning Tree (MST)

Graph-based clustering represents data points as nodes in a graph, with edges representing relationships or distance metrics between individual samples.

- **Complete Graph Construction:** Given a dataset $X = \{x_1, x_2, \dots, x_n\}$, each tissue sample is treated as a vertex. An edge weight $w(i, j)$

corresponds to the distance between sample i and sample j (typically Euclidean distance on scaled features): $w(i, j) = \sqrt{x_i - x_j}^2$

- **Minimum Spanning Tree (MST):** An MST is a subset of edges connecting all vertices in the graph without forming any cycles, while minimizing the total edge weight sum.
- **MST-Based Clustering Method:**
 1. Construct a fully connected graph with pairwise edge weights across standardized features.
 2. Compute the Minimum Spanning Tree (MST).
 3. To partition the data into K distinct clusters (such as Malignant and Benign groups), identify and remove the $K - 1$ largest (most expensive) edges from the MST.4. The resulting disconnected subgraphs represent the separate data clusters. ### Dataset Information Get the Breast Cancer Wisconsin (Diagnostic) dataset from Kaggle or via scikit-learn (load_breast_cancer). The dataset contains key cell nuclei features computed from digitized images of fine needle aspirates (FNA) of breast masses:
 - Total Instances: 569 samples
 - Target Classes: Malignant (M) and Benign (B)* Feature Attributes: 30 numerical features (e.g., mean radius, mean texture, mean perimeter, mean area, mean smoothness)

For 2D visualization, two key features (such as mean radius and mean texture) will be selected.

Task 1: Import Libraries and Load Dataset

- Import required libraries (numpy, pandas, matplotlib, scipy, networkx, sklearn).
- Load the Breast Cancer dataset using below link from Kaggle.

<https://www.kaggle.com/datasets/utkarshx27/breast-cancer-wisconsin-diagnostic-dataset>

```
In [ ]: # Task 1: Import Libraries and Load Dataset
```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy
import networkx as nx

from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import adjusted_rand_score

# Load Breast Cancer Dataset
data = load_breast_cancer()

# Convert dataset into DataFrame
df = pd.DataFrame(data.data, columns=data.feature_names)

# Add diagnosis column (0 = Malignant, 1 = Benign)
df['diagnosis'] = pd.Series(data.target).map({
    0: 'M',
    1: 'B'
})

# Display first five rows
print("First 5 Rows of Dataset:")
print(df.head())

# Display dataset shape
print("\nDataset Shape:", df.shape)

# Display number of samples and features
print("\nNumber of Samples:", df.shape[0])
print("Number of Features:", len(data.feature_names))

# Display diagnosis distribution
print("\nDiagnosis Distribution:")
print(df['diagnosis'].value_counts())

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Display dataset information
print("\nDataset Information:")
print(df.info())

```

First 5 Rows of Dataset:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness
0	17.99	10.38	122.80	1001.0	0.11840
1	20.57	17.77	132.90	1326.0	0.08474
2	19.69	21.25	130.00	1203.0	0.10960
3	11.42	20.38	77.58	386.1	0.14250
4	20.29	14.34	135.10	1297.0	0.10030

	mean compactness	mean concavity	mean concave points	mean symmetry
0	0.27760	0.3001	0.14710	0.2419
1	0.07864	0.0869	0.07017	0.1812
2	0.15990	0.1974	0.12790	0.2069
3	0.28390	0.2414	0.10520	0.2597
4	0.13280	0.1980	0.10430	0.1809

	mean fractal dimension	...	worst texture	worst perimeter	worst area
0	0.07871	...	17.33	184.60	201
1	0.05667	...	23.41	158.80	195
2	0.05999	...	25.53	152.50	170
3	0.09744	...	26.50	98.87	56
4	0.05883	...	16.67	152.20	157

	worst smoothness	worst compactness	worst concavity	worst concave points
0	0.1622	0.6656	0.7119	
1	0.1238	0.1866	0.2416	
2	0.1444	0.4245	0.4504	
3	0.2098	0.8663	0.6869	
4	0.1374	0.2050	0.4000	

	worst symmetry	worst fractal dimension	diagnosis
0	0.4601	0.11890	M
1	0.2750	0.08902	M
2	0.3613	0.08758	M
3	0.6638	0.17300	M
4	0.2364	0.07678	M

[5 rows x 31 columns]

Dataset Shape: (569, 31)

Number of Samples: 569

Number of Features: 30

Diagnosis Distribution:

diagnosis

B 357

M 212

Name: count, dtype: int64

Missing Values:

mean radius	0
mean texture	0
mean perimeter	0
mean area	0
mean smoothness	0
mean compactness	0
mean concavity	0
mean concave points	0
mean symmetry	0
mean fractal dimension	0
radius error	0
texture error	0
perimeter error	0
area error	0
smoothness error	0
compactness error	0
concavity error	0
concave points error	0
symmetry error	0
fractal dimension error	0
worst radius	0
worst texture	0
worst perimeter	0
worst area	0
worst smoothness	0
worst compactness	0
worst concavity	0
worst concave points	0
worst symmetry	0
worst fractal dimension	0
diagnosis	0

dtype: int64

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 569 entries, 0 to 568

Data columns (total 31 columns):

#	Column	Non-Null Count	Dtype
0	mean radius	569 non-null	float64
1	mean texture	569 non-null	float64
2	mean perimeter	569 non-null	float64
3	mean area	569 non-null	float64
4	mean smoothness	569 non-null	float64
5	mean compactness	569 non-null	float64
6	mean concavity	569 non-null	float64

```

7   mean concave points      569 non-null    float64
8   mean symmetry           569 non-null    float64
9   mean fractal dimension  569 non-null    float64
10  radius error            569 non-null    float64
11  texture error           569 non-null    float64
12  perimeter error         569 non-null    float64
13  area error              569 non-null    float64
14  smoothness error        569 non-null    float64
15  compactness error       569 non-null    float64
16  concavity error         569 non-null    float64
17  concave points error    569 non-null    float64
18  symmetry error          569 non-null    float64
19  fractal dimension error  569 non-null    float64
20  worst radius            569 non-null    float64
21  worst texture           569 non-null    float64
22  worst perimeter         569 non-null    float64
23  worst area              569 non-null    float64
24  worst smoothness        569 non-null    float64
25  worst compactness       569 non-null    float64
26  worst concavity         569 non-null    float64
27  worst concave points    569 non-null    float64
28  worst symmetry          569 non-null    float64
29  worst fractal dimension  569 non-null    float64
30  diagnosis               569 non-null    object
dtypes: float64(30), object(1)
memory usage: 137.9+ KB
None

```

Task 2: Explore the Dataset

- Display missing value checks, summary statistics, and feature dimensions.
- Check the diagnosis class distribution (Malignant vs. Benign).

```

In [ ]: # Task 2: Explore the Dataset

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer

# Load Breast Cancer Dataset
data = load_breast_cancer()

df = pd.DataFrame(data.data, columns=data.feature_names)

# Add diagnosis column: M = Malignant, B = Benign

```

```

df['diagnosis'] = pd.Series(data.target).map({0: 'M', 1: 'B'})

# 1. Display first five rows
print("First 5 Rows:")
print(df.head())

# 2. Check dataset dimensions
print("\nDataset Dimensions:", df.shape)
print("Number of Samples:", df.shape[0])
print("Number of Features:", df.shape[1] - 1)

# 3. Check missing values
print("\nMissing Values in Each Column:")
print(df.isnull().sum())

print("\nTotal Missing Values:", df.isnull().sum().sum())

# 4. Display summary statistics
print("\nSummary Statistics:")
print(df.describe())

# 5. Check data types
print("\nColumn Data Types:")
print(df.dtypes)

# 6. Check diagnosis class distribution
print("\nDiagnosis Class Distribution:")
print(df['diagnosis'].value_counts())

# 7. Display diagnosis percentages
print("\nDiagnosis Distribution (%):")
print((df['diagnosis'].value_counts(normalize=True) * 100).round(2))

# 8. Visualize diagnosis distribution
sns.countplot(data=df, x='diagnosis')
plt.title("Breast Cancer Diagnosis Distribution")
plt.xlabel("Diagnosis (M = Malignant, B = Benign)")
plt.ylabel("Number of Samples")
plt.show()

```

First 5 Rows:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness
0	17.99	10.38	122.80	1001.0	0.11840
1	20.57	17.77	132.90	1326.0	0.08474
2	19.69	21.25	130.00	1203.0	0.10960
3	11.42	20.38	77.58	386.1	0.14250
4	20.29	14.34	135.10	1297.0	0.10030

	mean compactness	mean concavity	mean concave points	mean symmetry
0	0.27760	0.3001	0.14710	0.2419
1	0.07864	0.0869	0.07017	0.1812
2	0.15990	0.1974	0.12790	0.2069
3	0.28390	0.2414	0.10520	0.2597
4	0.13280	0.1980	0.10430	0.1809

	mean fractal dimension	...	worst texture	worst perimeter	worst area
0	0.07871	...	17.33	184.60	201
1	0.05667	...	23.41	158.80	195
2	0.05999	...	25.53	152.50	170
3	0.09744	...	26.50	98.87	56
4	0.05883	...	16.67	152.20	157

	worst smoothness	worst compactness	worst concavity	worst concave points
0	0.1622	0.6656	0.7119	
1	0.1238	0.1866	0.2416	
2	0.1444	0.4245	0.4504	
3	0.2098	0.8663	0.6869	
4	0.1374	0.2050	0.4000	

	worst symmetry	worst fractal dimension	diagnosis
0	0.4601	0.11890	M
1	0.2750	0.08902	M
2	0.3613	0.08758	M
3	0.6638	0.17300	M
4	0.2364	0.07678	M

[5 rows x 31 columns]

Dataset Dimensions: (569, 31)

Number of Samples: 569

Number of Features: 30

Missing Values in Each Column:

```

mean radius      0
mean texture     0
mean perimeter   0
mean area        0
mean smoothness  0
mean compactness 0
mean concavity   0
mean concave points 0
mean symmetry    0
mean fractal dimension 0
radius error     0
texture error    0
perimeter error  0
area error       0
smoothness error 0
compactness error 0
concavity error  0
concave points error 0
symmetry error   0
fractal dimension error 0
worst radius     0
worst texture    0
worst perimeter  0
worst area       0
worst smoothness 0
worst compactness 0
worst concavity  0
worst concave points 0
worst symmetry   0
worst fractal dimension 0
diagnosis        0
dtype: int64

```

Total Missing Values: 0

Summary Statistics:

	mean radius	mean texture	mean perimeter	mean area \
count	569.000000	569.000000	569.000000	569.000000
mean	14.127292	19.289649	91.969033	654.889104
std	3.524049	4.301036	24.298981	351.914129
min	6.981000	9.710000	43.790000	143.500000
25%	11.700000	16.170000	75.170000	420.300000
50%	13.370000	18.840000	86.240000	551.100000
75%	15.780000	21.800000	104.100000	782.700000
max	28.110000	39.280000	188.500000	2501.000000

	mean smoothness	mean compactness	mean concavity	mean concave po
ints \				
count	569.000000	569.000000	569.000000	569.00
0000				
mean	0.096360	0.104341	0.088799	0.04
8919				
std	0.014064	0.052813	0.079720	0.03

8803				
min	0.052630	0.019380	0.000000	0.00
0000				
25%	0.086370	0.064920	0.029560	0.02
0310				
50%	0.095870	0.092630	0.061540	0.03
3500				
75%	0.105300	0.130400	0.130700	0.07
4000				
max	0.163400	0.345400	0.426800	0.20
1200				

	mean symmetry	mean fractal dimension	...	worst radius \
count	569.000000	569.000000	...	569.000000
mean	0.181162	0.062798	...	16.269190
std	0.027414	0.007060	...	4.833242
min	0.106000	0.049960	...	7.930000
25%	0.161900	0.057700	...	13.010000
50%	0.179200	0.061540	...	14.970000
75%	0.195700	0.066120	...	18.790000
max	0.304000	0.097440	...	36.040000

	worst texture	worst perimeter	worst area	worst smoothness \
count	569.000000	569.000000	569.000000	569.000000
mean	25.677223	107.261213	880.583128	0.132369
std	6.146258	33.602542	569.356993	0.022832
min	12.020000	50.410000	185.200000	0.071170
25%	21.080000	84.110000	515.300000	0.116600
50%	25.410000	97.660000	686.500000	0.131300
75%	29.720000	125.400000	1084.000000	0.146000
max	49.540000	251.200000	4254.000000	0.222600

	worst compactness	worst concavity	worst concave points \
count	569.000000	569.000000	569.000000
mean	0.254265	0.272188	0.114606
std	0.157336	0.208624	0.065732
min	0.027290	0.000000	0.000000
25%	0.147200	0.114500	0.064930
50%	0.211900	0.226700	0.099930
75%	0.339100	0.382900	0.161400
max	1.058000	1.252000	0.291000

	worst symmetry	worst fractal dimension
count	569.000000	569.000000
mean	0.290076	0.083946
std	0.061867	0.018061
min	0.156500	0.055040
25%	0.250400	0.071460
50%	0.282200	0.080040
75%	0.317900	0.092080
max	0.663800	0.207500

[8 rows x 30 columns]

Column Data Types:

mean radius	float64
mean texture	float64
mean perimeter	float64
mean area	float64
mean smoothness	float64
mean compactness	float64
mean concavity	float64
mean concave points	float64
mean symmetry	float64
mean fractal dimension	float64
radius error	float64
texture error	float64
perimeter error	float64
area error	float64
smoothness error	float64
compactness error	float64
concavity error	float64
concave points error	float64
symmetry error	float64
fractal dimension error	float64
worst radius	float64
worst texture	float64
worst perimeter	float64
worst area	float64
worst smoothness	float64
worst compactness	float64
worst concavity	float64
worst concave points	float64
worst symmetry	float64
worst fractal dimension	float64
diagnosis	object

dtype: object

Diagnosis Class Distribution:

diagnosis

B 357

M 212

Name: count, dtype: int64

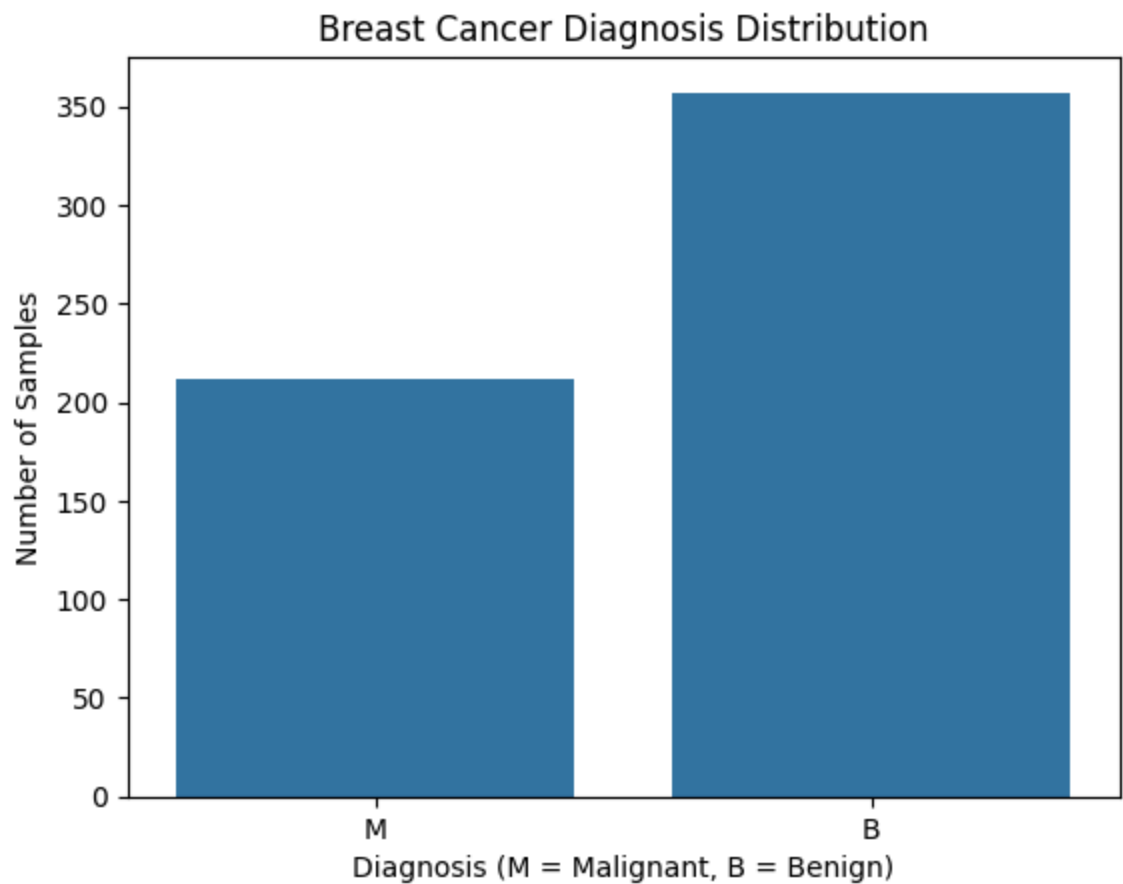
Diagnosis Distribution (%):

diagnosis

B 62.74

M 37.26

Name: proportion, dtype: float64



Task 3: Preprocess and Scale Features

- Select numerical features and drop non-informative identifiers (such as id).
- Apply StandardScaler to normalize feature magnitudes prior to graph distance calculation.

```
In [ ]: import pandas as pd
        from sklearn.datasets import load_breast_cancer
        from sklearn.preprocessing import StandardScaler

        # Load Breast Cancer dataset
        data = load_breast_cancer()
        df = pd.DataFrame(data.data, columns=data.feature_names)

        # Select numerical features
        X = df.select_dtypes(include=['number'])

        # Check for non-informative identifier columns
        X = X.drop(columns=['id', 'Unnamed: 32'], errors='ignore')
```

```
# Apply StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Display results
print("Original Feature Dimensions:", X.shape)
print("Scaled Feature Dimensions:", X_scaled.shape)

print("\nFirst 5 Rows of Original Features:")
print(X.head())

print("\nFirst 5 Rows of Scaled Features:")
print(pd.DataFrame(X_scaled, columns=X.columns).head())

print("\nMean of Scaled Features (approximately 0):")
print(X_scaled.mean(axis=0).round(4))

print("\nStandard Deviation of Scaled Features (approximately 1):")
print(X_scaled.std(axis=0).round(4))

print("\nPreprocessing and scaling completed successfully!")
```

Original Feature Dimensions: (569, 30)

Scaled Feature Dimensions: (569, 30)

First 5 Rows of Original Features:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness
\					
0	17.99	10.38	122.80	1001.0	0.11840
1	20.57	17.77	132.90	1326.0	0.08474
2	19.69	21.25	130.00	1203.0	0.10960
3	11.42	20.38	77.58	386.1	0.14250
4	20.29	14.34	135.10	1297.0	0.10030

	mean compactness	mean concavity	mean concave points	mean symmetry
\				
0	0.27760	0.3001	0.14710	0.2419
1	0.07864	0.0869	0.07017	0.1812
2	0.15990	0.1974	0.12790	0.2069
3	0.28390	0.2414	0.10520	0.2597
4	0.13280	0.1980	0.10430	0.1809

	mean fractal dimension	...	worst radius	worst texture	worst perimeter
\					
0	0.07871	...	25.38	17.33	18
4.60					
1	0.05667	...	24.99	23.41	15
8.80					
2	0.05999	...	23.57	25.53	15
2.50					
3	0.09744	...	14.91	26.50	9
8.87					
4	0.05883	...	22.54	16.67	15
2.20					

	worst area	worst smoothness	worst compactness	worst concavity	\
0	2019.0	0.1622	0.6656	0.7119	
1	1956.0	0.1238	0.1866	0.2416	
2	1709.0	0.1444	0.4245	0.4504	
3	567.7	0.2098	0.8663	0.6869	
4	1575.0	0.1374	0.2050	0.4000	

	worst concave points	worst symmetry	worst fractal dimension
0	0.2654	0.4601	0.11890
1	0.1860	0.2750	0.08902
2	0.2430	0.3613	0.08758
3	0.2575	0.6638	0.17300
4	0.1625	0.2364	0.07678

[5 rows x 30 columns]

First 5 Rows of Scaled Features:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness
\					
0	1.097064	-2.073335	1.269934	0.984375	1.568466
1	1.829821	-0.353632	1.685955	1.908708	-0.826962
2	1.579888	0.456187	1.566503	1.558884	0.942210

3	-0.768909	0.253732	-0.592687	-0.764464	3.283553
4	1.750297	-1.151816	1.776573	1.826229	0.280372

	mean compactness	mean concavity	mean concave points	mean symmetry
\				
0	3.283515	2.652874	2.532475	2.217515
1	-0.487072	-0.023846	0.548144	0.001392
2	1.052926	1.363478	2.037231	0.939685
3	3.402909	1.915897	1.451707	2.867383
4	0.539340	1.371011	1.428493	-0.009560

	mean fractal dimension	...	worst radius	worst texture	worst perimeter
\					
0	2.255747	...	1.886690	-1.359293	2.303601
1	-0.868652	...	1.805927	-0.369203	1.535126
2	-0.398008	...	1.511870	-0.023974	1.347475
3	4.910919	...	-0.281464	0.133984	-0.249939
4	-0.562450	...	1.298575	-1.466770	1.338539

	worst area	worst smoothness	worst compactness	worst concavity
\				
0	2.001237	1.307686	2.616665	2.109526
1	1.890489	-0.375612	-0.430444	-0.146749
2	1.456285	0.527407	1.082932	0.854974
3	-0.550021	3.394275	3.893397	1.989588
4	1.220724	0.220556	-0.313395	0.613179

	worst concave points	worst symmetry	worst fractal dimension
\			
0	2.296076	2.750622	1.937015
1	1.087084	-0.243890	0.281190
2	1.955000	1.152255	0.201391
3	2.175786	6.046041	4.935010
4	0.729259	-0.868353	-0.397100

[5 rows x 30 columns]

Mean of Scaled Features (approximately 0):

```
[-0.  0. -0. -0. -0.  0.  0. -0.  0.  0. -0. -0. -0. -0.  0.  0.  0.
  0. -0. -0.  0. -0.  0. -0. -0.  0.  0.  0. -0.]
```

Standard Deviation of Scaled Features (approximately 1):

```
[1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.
  1.  1.  1.  1.  1.  1.]
```

Preprocessing and scaling completed successfully!

Task 4: Construct Pairwise Distance Matrix & Graph

- Compute pairwise Euclidean distance matrix across all scaled samples using `scipy.spatial.distance.pdist` or `cdist`.
- Represent the dataset as a weighted complete graph.

```
In [ ]: import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
from scipy.spatial.distance import pdist, squareform

# Compute pairwise Euclidean distances
distance_matrix = squareform(pdist(X_scaled, metric='euclidean'))

print("Pairwise Distance Matrix Shape:", distance_matrix.shape)
print("\nFirst 5x5 Distance Matrix:")
print(np.round(distance_matrix[:5, :5], 2))

# Construct weighted complete graph
G = nx.Graph()
n = X_scaled.shape[0]

# Add nodes for all samples
G.add_nodes_from(range(n))

# Add weighted edges between every pair of samples
for i in range(n):
    for j in range(i + 1, n):
        G.add_edge(i, j, weight=distance_matrix[i, j])

print("\nNumber of Nodes:", G.number_of_nodes())
print("Number of Edges:", G.number_of_edges())
print("Is Graph Complete:", nx.is_connected(G) and
      G.number_of_edges() == n * (n - 1) // 2)

# Visualize a small sample of the complete graph
sample_size = 30
G_sample = G.subgraph(range(sample_size))

plt.figure(figsize=(10, 7))
pos = nx.spring_layout(G_sample, seed=42)
nx.draw(
    G_sample, pos,
    node_size=100,
    node_color='skyblue',
    edge_color='gray',
    with_labels=True,
```

```

    font_size=7
)
plt.title("Weighted Complete Graph (First 30 Samples)")
plt.show()

```

Pairwise Distance Matrix Shape: (569, 569)

First 5x5 Distance Matrix:

```

[[ 0.    10.32  6.78 10.47  8.67]
 [10.32  0.    5.03 16.25  4.38]
 [ 6.78  5.03  0.   12.84  4.46]
 [10.47 16.25 12.84  0.   15.38]
 [ 8.67  4.38  4.46 15.38  0.   ]]

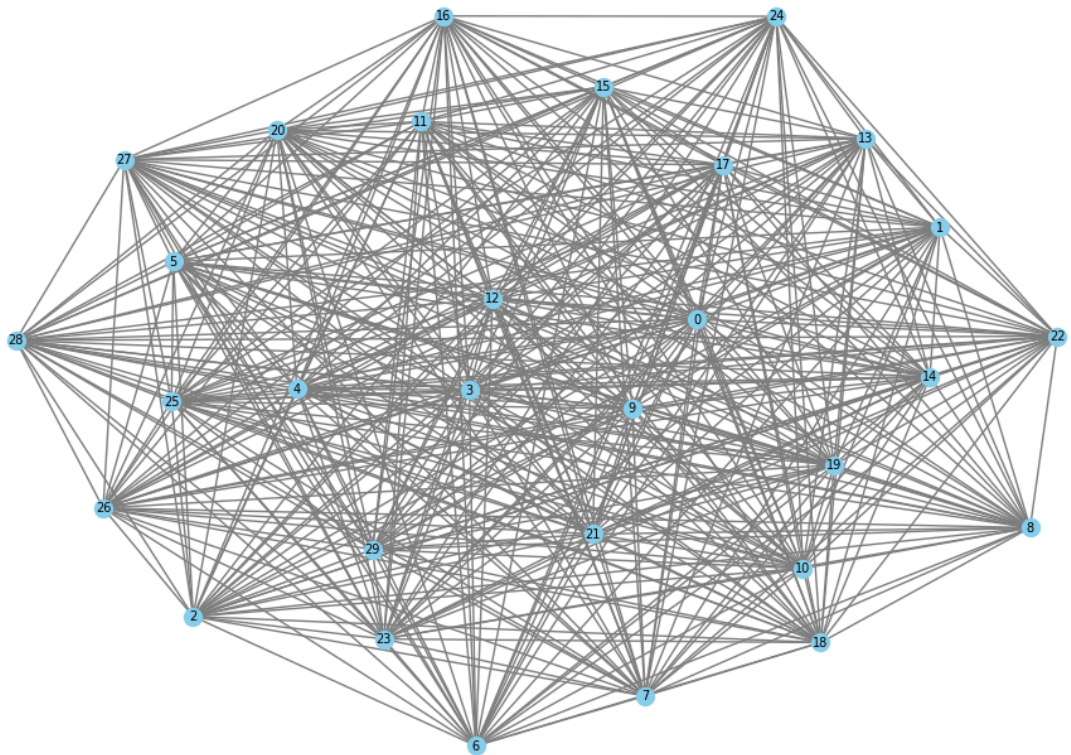
```

Number of Nodes: 569

Number of Edges: 161596

Is Graph Complete: True

Weighted Complete Graph (First 30 Samples)



Task 5: Compute the Minimum Spanning Tree (MST)

- Compute the MST using `scipy.sparse.csgraph.minimum_spanning_tree` or

networkx.minimum_spanning_tree.

- Extract the resulting graph connections and weights.

```
In [ ]: import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
from scipy.sparse.csgraph import minimum_spanning_tree

# Compute Minimum Spanning Tree from the distance matrix
mst_matrix = minimum_spanning_tree(distance_matrix)

# Convert MST matrix to NetworkX graph
MST = nx.from_scipy_sparse_array(mst_matrix)

# Extract MST connections and weights
mst_edges = list(MST.edges(data=True))

print("Minimum Spanning Tree (MST) Computed Successfully!")
print("Number of Nodes:", MST.number_of_nodes())
print("Number of Edges:", MST.number_of_edges())

print("\nFirst 10 MST Connections and Weights:")
for u, v, data in mst_edges[:10]:
    print(f"Node {u} -- Node {v}, Weight: {data['weight']:.4f}")

# Calculate total MST weight
total_weight = sum(data['weight'] for _, _, data in mst_edges)
print("\nTotal MST Weight:", round(total_weight, 4))

# Visualize the MST
plt.figure(figsize=(12, 8))
pos = nx.spring_layout(MST, seed=42)

nx.draw_networkx_nodes(MST, pos, node_size=20, node_color='skyblue')
nx.draw_networkx_edges(MST, pos, edge_color='gray', width=0.7)

plt.title("Minimum Spanning Tree (MST) - Breast Cancer Dataset")
plt.axis("off")
plt.show()
```

Minimum Spanning Tree (MST) Computed Successfully!

Number of Nodes: 569

Number of Edges: 568

First 10 MST Connections and Weights:

Node 0 -- Node 77, Weight: 4.8299

Node 1 -- Node 365, Weight: 2.2871

Node 2 -- Node 45, Weight: 2.6763

Node 2 -- Node 77, Weight: 3.7056

Node 2 -- Node 535, Weight: 2.4515

Node 3 -- Node 190, Weight: 7.8710

Node 4 -- Node 498, Weight: 2.8827

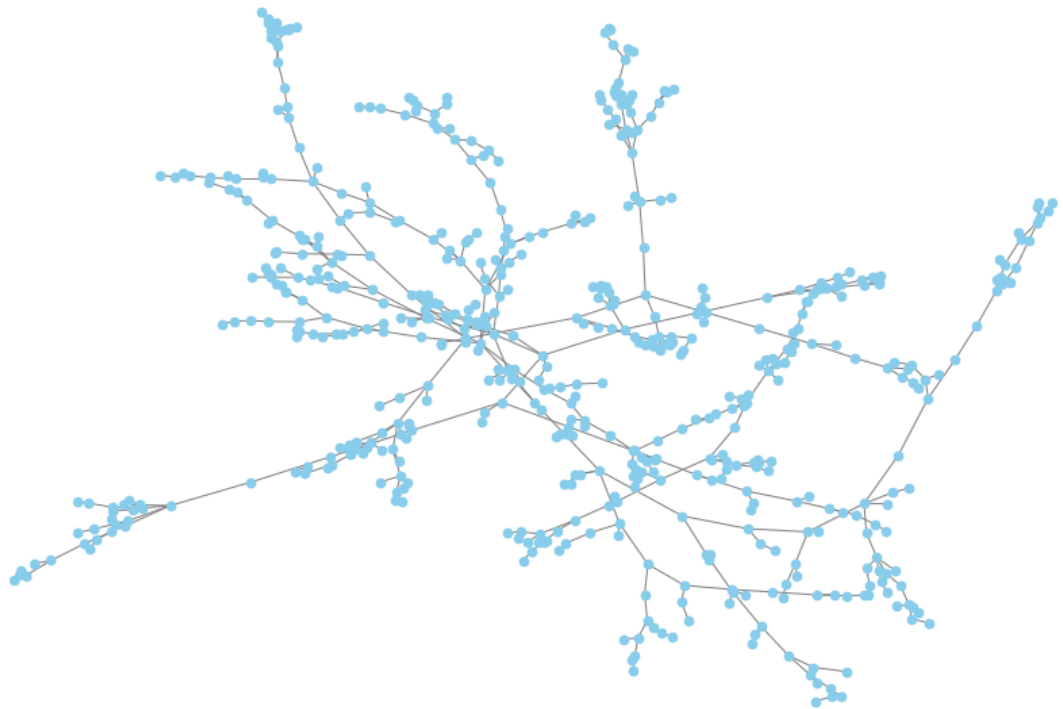
Node 5 -- Node 8, Weight: 2.6951

Node 5 -- Node 31, Weight: 3.0699

Node 5 -- Node 47, Weight: 2.5029

Total MST Weight: 1393.8521

Minimum Spanning Tree (MST) - Breast Cancer Dataset



Task 6: Implement Graph-Based Clustering (\$K=2\$)

- Sort the edges of the MST in descending order by edge weight.
- Remove the largest $K - 1$ edge (1 largest edge for 2 clusters) to split the

MST into 2 connected components.

- Assign final cluster labels based on the resulting component indices.

```
In [ ]: import numpy as np
import pandas as pd
import networkx as nx
from sklearn.datasets import load_breast_cancer

# Load dataset with diagnosis labels
data = load_breast_cancer()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['diagnosis'] = pd.Series(data.target).map({0: 'M', 1: 'B'})

# Copy the MST
G_cluster = MST.copy()

# Sort MST edges in descending order by weight
sorted_edges = sorted(
    G_cluster.edges(data=True),
    key=lambda edge: edge[2]['weight'],
    reverse=True
)

# Remove the largest edge to create K=2 clusters
u, v, edge_data = sorted_edges[0]
print("Largest MST Edge:")
print(f"Node {u} -- Node {v}, Weight: {edge_data['weight']:.4f}")

G_cluster.remove_edge(u, v)

# Find connected components
components = list(nx.connected_components(G_cluster))

# Assign cluster labels
cluster_labels = np.zeros(len(df), dtype=int)

for cluster_id, component in enumerate(components):
    for node in component:
        cluster_labels[node] = cluster_id

# Add cluster labels to the dataset
df['Cluster'] = cluster_labels

# Display results
print("\nNumber of Clusters:", len(components))
print("\nCluster Sizes:")
print(df['Cluster'].value_counts().sort_index())

print("\nFirst 20 Cluster Labels:")
print(cluster_labels[:20])

print("\nDataset with Cluster Labels:")
print(df[['diagnosis', 'Cluster']].head(10))
```

```
print("\nDiagnosis vs Cluster:")
print(pd.crosstab(df['diagnosis'], df['Cluster']))

print("\nGraph-Based Clustering Completed Successfully!")
```

Largest MST Edge:
Node 461 -- Node 503, Weight: 12.2999

Number of Clusters: 2

Cluster Sizes:
Cluster
0 567
1 2
Name: count, dtype: int64

First 20 Cluster Labels:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]

Dataset with Cluster Labels:

	diagnosis	Cluster
0	M	0
1	M	0
2	M	0
3	M	0
4	M	0
5	M	0
6	M	0
7	M	0
8	M	0
9	M	0

Diagnosis vs Cluster:

Cluster	0	1
diagnosis		
B	357	0
M	210	2

Graph-Based Clustering Completed Successfully!

Task 7: evaluate MST Clustering Performance

Evaluate cluster assignment against ground truth labels using:

- Silhouette Score
- Adjusted Rand Index (ARI)

- Normalized Mutual Information (NMI)

```
In [ ]: import numpy as np
from sklearn.metrics import silhouette_score, adjusted_rand_score, normalized_mutual_info_score
from sklearn.datasets import load_breast_cancer

# Load ground-truth diagnosis labels
data = load_breast_cancer()
y_true = data.target

# Use cluster_labels from Task 6
y_pred = cluster_labels

# Calculate evaluation metrics
sil_score = silhouette_score(X_scaled, y_pred)
ari_score = adjusted_rand_score(y_true, y_pred)
nmi_score = normalized_mutual_info_score(y_true, y_pred)

# Display results
print("MST Clustering Performance Evaluation")
print("-----")
print("Silhouette Score:", round(sil_score, 4))
print("Adjusted Rand Index (ARI):", round(ari_score, 4))
print("Normalized Mutual Information (NMI):", round(nmi_score, 4))
```

```
MST Clustering Performance Evaluation
-----
Silhouette Score: 0.6607
Adjusted Rand Index (ARI): 0.0048
Normalized Mutual Information (NMI): 0.0102
```

Task 8: Visualize MST and Graph-Based Clusters

- Select two representative features (e.g., mean radius vs. mean texture) for 2D plotting.
 - Plot 1: Unclustered standardized data points.
- Plot 2: Minimum Spanning Tree (MST) edge connections overlaid on the data.
- Plot 3: Final clusters formed after cutting the $K - 1$ largest edge(s).

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
import networkx as nx
from sklearn.datasets import load_breast_cancer

# Load dataset
data = load_breast_cancer()
```

```

feature_names = list(data.feature_names)

# Select mean radius and mean texture
radius_idx = feature_names.index('mean radius')
texture_idx = feature_names.index('mean texture')

# Extract standardized features
X_2d = X_scaled[:, [radius_idx, texture_idx]]

# Plot 1: Unclustered standardized data
plt.figure(figsize=(8, 6))
plt.scatter(X_2d[:, 0], X_2d[:, 1], s=25, alpha=0.7)
plt.title("Plot 1: Unclustered Standardized Data")
plt.xlabel("Mean Radius (Standardized)")
plt.ylabel("Mean Texture (Standardized)")
plt.grid(True, alpha=0.3)
plt.show()

# Plot 2: MST overlaid on standardized data
plt.figure(figsize=(10, 7))

for u, v in MST.edges():
    plt.plot(
        [X_2d[u, 0], X_2d[v, 0]],
        [X_2d[u, 1], X_2d[v, 1]],
        color='gray', linewidth=0.7, alpha=0.6
    )

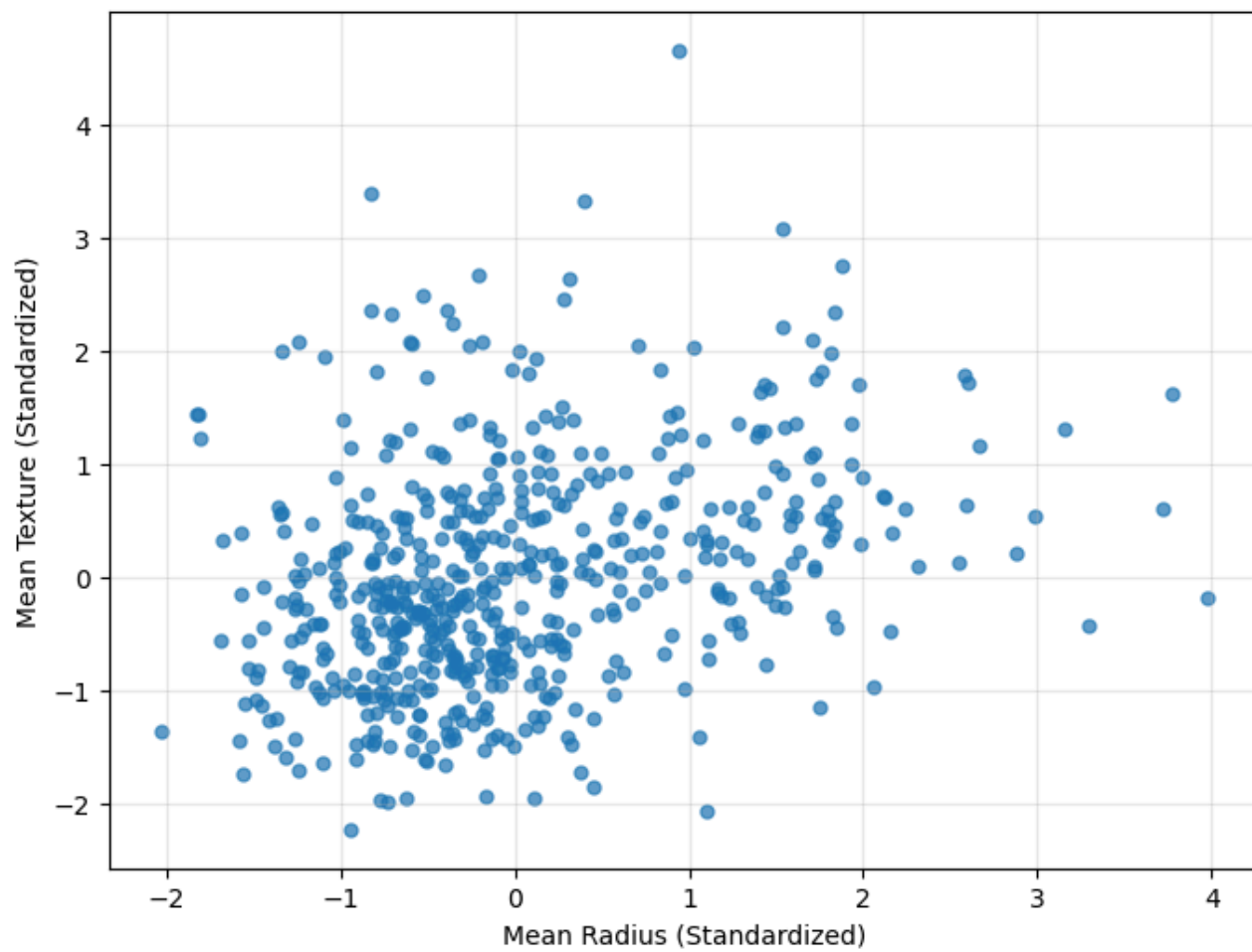
plt.scatter(X_2d[:, 0], X_2d[:, 1], s=25, color='blue', zorder=2)
plt.title("Plot 2: Minimum Spanning Tree (MST)")
plt.xlabel("Mean Radius (Standardized)")
plt.ylabel("Mean Texture (Standardized)")
plt.grid(True, alpha=0.3)
plt.show()

# Plot 3: Final clusters after removing the largest MST edge
plt.figure(figsize=(8, 6))
scatter = plt.scatter(
    X_2d[:, 0], X_2d[:, 1],
    c=cluster_labels, cmap='viridis',
    s=35, alpha=0.8
)

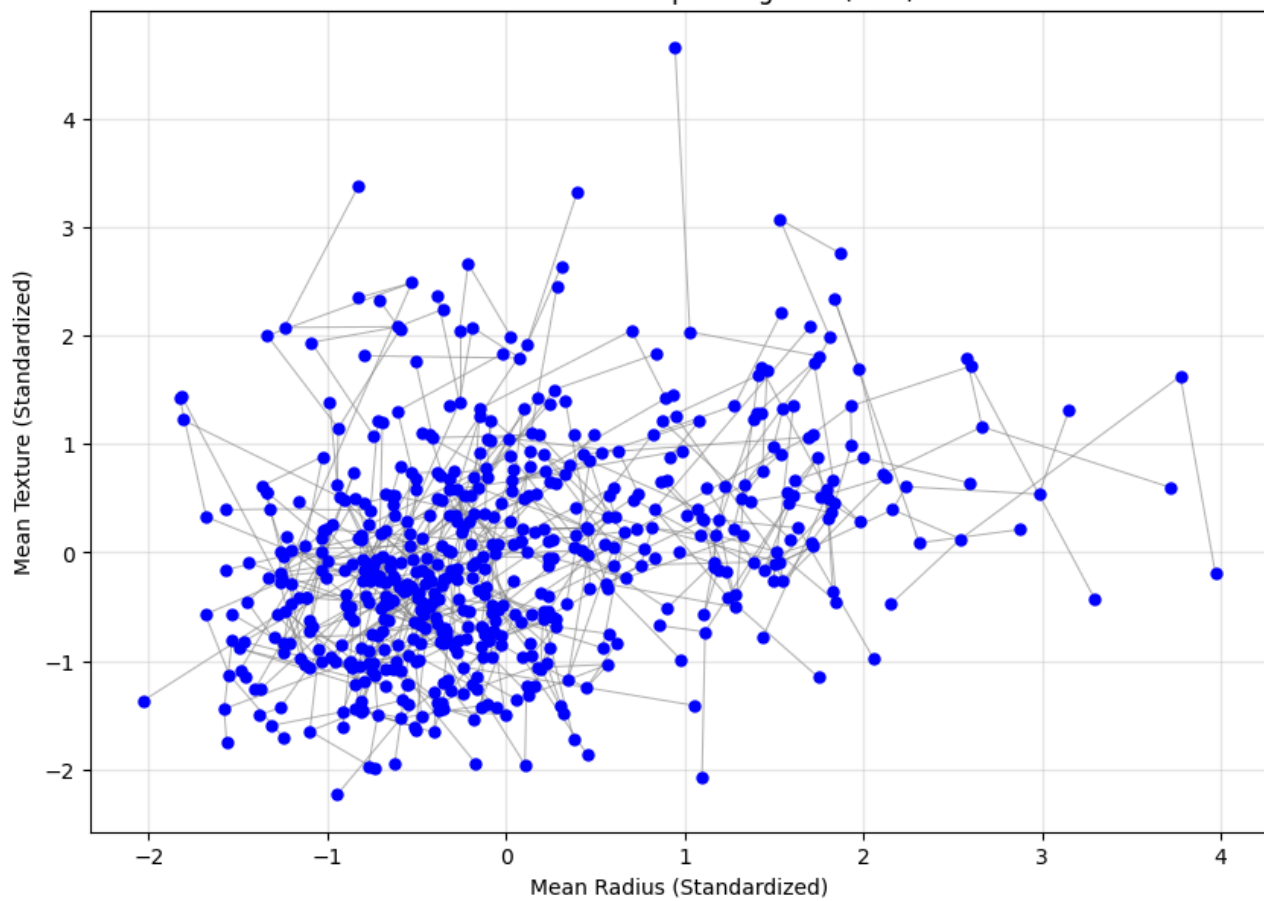
plt.title("Plot 3: MST Graph-Based Clusters (K=2)")
plt.xlabel("Mean Radius (Standardized)")
plt.ylabel("Mean Texture (Standardized)")
plt.legend(*scatter.legend_elements(), title="Cluster")
plt.grid(True, alpha=0.3)
plt.show()

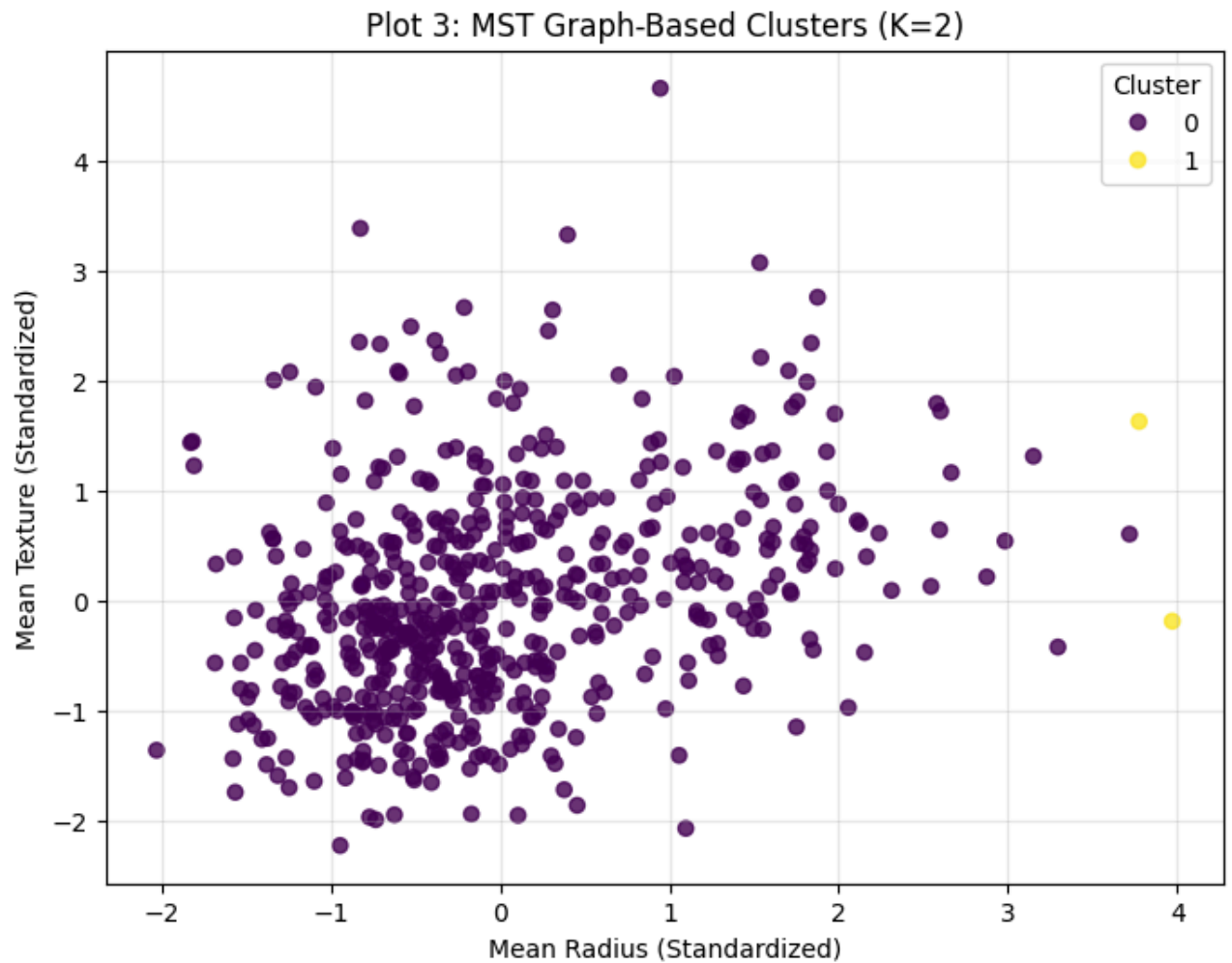
```

Plot 1: Unclustered Standardized Data



Plot 2: Minimum Spanning Tree (MST)





Conclusion

The experiment successfully demonstrated the implementation of Graph-Based Clustering using a Minimum Spanning Tree (MST) on the Breast Cancer Wisconsin (Diagnostic) dataset. The dataset was preprocessed, cleaned, and standardized before computing pairwise distances. The MST was generated, and by removing the largest edge(s), clusters representing tumor categories were obtained, evaluated using clustering metrics, and visualized.